

Architecture

BANK-SPEC is one Clojure application running in memory. Three modules own separate immutable histories and communicate through explicit function ports. The design prioritizes explainable financial behavior and deterministic verification. The detailed architecture retains implementation and calculation notes.

Responsibilities and boundaries

- **Authorization** owns available funds, holds, operational snapshots and decisions. It records identity, account version and outcome atomically before making an event available for delivery.
- **Ledger** owns balanced journal entries and accounting balances. Each accepted financial movement posts once by transaction ID. Booking date, value date and journal position make historical queries reproducible.
- **Yield and Fees** owns dated assessments, interest components, financial intents and settlement receipts. It calculates from approved events, saves the complete intended command, then requests its financial effect through Authorization.

Each domain has `logic`, `ports`, `db` and `model` directories, mirrored in tests. Logic contains pure functions only. `ports/api_server.clj` coordinates public operations; `ports/api_client.clj` owns outgoing calls. `db/memory.clj` owns local atomic state replacement. `model/models.clj` describes internal data without changing public validation. Ledger has no outgoing dependency. Shared arithmetic and validation also remain pure.

Recording, delivery and recovery

A technical entry point submits transactions to Authorization. The deterministic dispatcher sends saved envelopes to Yield and, for financial movements, Ledger. It acknowledges each destination independently. Neither Authorization nor Yield reads Ledger balances.

Local locks protect identity, version and state changes; no cross-module call runs under those locks. Delivery can lag, so operational balances and accounting balances have different observation boundaries. Reports expose known pending work and never trigger recovery as a side effect of reading.

Yield saves the exact payment or fee command before submission. A lost response leaves that command pending for an identical retry. A confirmed result, confirmed duplicate or delivered event reconciles its original assessment or component receipt before later settlement selection. A different settlement cannot pay the same component again. A definite rejection permits recalculation; uncertainty does not. This recovery works only while the process and its memory remain alive.

Time and immutable corrections

Economic value date, recording date and observed event/journal position remain distinct. Late inputs append linked differences; they never rewrite earlier payments, decisions or captured reports. Pending interest is separate from posted money until settlement. Exact decimal arithmetic and explicit currency rounding implement the adopted numerical policy.

The executable checks cover module boundaries, failure recovery and the supplied replay against an independent numerical oracle. Tradeoffs explains the deliberate limitations; production considerations identifies the controls and structural changes needed before live use.

Tradeoffs

The design favors a small, explainable financial model. The following choices are implemented; production considerations identifies additional work, not delivered capabilities.

Choice	Benefit	Cost or boundary
One process, three modules, explicit ports	Separates financial ownership without distributed infrastructure.	Shared failure domain and memory limit. Ports do not provide security or process isolation.
Pure logic and local atomic transitions	Tests can exercise decisions without adapters; each module records identity and state together.	Local locks serialize changes. Yield jobs require serial execution.
Append-only records and corrections	Preserves previous decisions and explains changed historical balances.	History and indexes grow; dated queries and recalculations scan retained records.
Save financial intent before submission	Unknown outcomes preserve the original command; confirmation prevents duplicate payment.	A pending command blocks another financial operation for that account. Recovery does not survive process exit.
Recorded identity wins without comparing payload	Retries return the original outcome without another effect.	A reused ID can hide different content. The separate failing challenge demonstrates this accepted limitation.
Reconstruct balances using booking and value dates	Makes late corrections reproducible.	More scanning; internal counter completeness does not prove all external inputs arrived.
Exact decimal amounts and daily currency rounding	Implements the adopted numerical rules with explainable reconciliation.	HALF_UP, rates and fee configuration remain product choices, not consequences of the currency code.

What was cut and why

Durable storage, HTTP, UI and a broker were excluded by the exercise. Automatic expiry, a real business calendar, FX, taxes and payment-network integration lack required rules in this scope. These exclusions keep the implementation defensible but leave crash recovery, external reconciliation and contractual timing unresolved for production.

Unsafe submission-before-recording and incomplete financial metadata were corrected, with regressions retained. Scope exclusions were not systems built and discarded. AMBIGUITIES and REJECTED preserve the adopted decisions and actual rejected approaches.

Production Considerations

These are limits and recommendations, not implemented production capabilities.

Growth at 100 times the volume

The first avoidable bottleneck is delivery lookup: each drain scans retained `:delivery-order`, including acknowledged IDs. Acknowledgement removes the envelope from `:deliveries`; order and acknowledgement IDs remain. Draining after every transaction can therefore accumulate quadratic work.

Unbounded state includes Authorization's identity results, decision/snapshot history and delivery indexes; Ledger's journal and identity set; and Yield's events, fee/interest components, receipts, intent history and identity indexes. Ledger queries scan history; Yield repeatedly scans events/components. At 100× retained records, a linear scan examines approximately 100× records. This is an algorithmic estimate, not a benchmark; Clojure collections share structure.

Cheapest change: maintain a separate ordered index of pending deliveries, removing acknowledged entries from that work index while preserving financial/audit history. Next, add per-account journal indexes and dated checkpoints that retain both booking and value boundaries. These improve scans but do not bound history. Durable storage and a governed archive policy would still be required; deleting old identities or settled components can break duplicate protection or corrections.

Authorization endings and intermediate states

These behaviors are adopted project rules. Reservation changes affect availability; confirmed settlements move money.

Situation	Model behavior
Purchase authorization lacks available funds	Record a permanent decline, with no hold, posting or new snapshot. Later funds do not reactivate it.
Merchant cancels the remaining purchase	Full explicit release ends the hold; restore availability without crediting money.
Final capture is below the reservation	Debit the capture, release the unused remainder and end the hold without a release credit.
Capture equals or exceeds the remaining hold	Debit the entire confirmed amount and exhaust the hold, including with a non-final flag. Insufficient availability does not suppress confirmed money.
Capture arrives after decline, release or exhaustion	Record the debit and <code>inactive-authorization</code> ; create no new hold. An unknown reference instead produces <code>missing-authorization</code> .
A captured payment is reversed	Append the principal reversal. Do not recreate the consumed/released hold; preserve any residual hold. Corrections follow their own rules.

Partial release and non-final capture below the remaining hold are intermediate states: free or consume only the specified amount, retaining the remainder. With no further event, an approved residual hold stays active. No expiry policy or timer is implemented; a production timeout needs a contractual deadline and an audited release. Invalid releases leave state unchanged; duplicate transaction IDs repeat no effect.

Value dates for a UAE licensed bank

A late booking can change an earlier economic balance and its fee/interest entitlement while preserving the previously observed operational report. Booking date, value date and observation boundary must remain distinguishable in reconciliation and customer explanations.

For consumer deposit products, CBUAE Standards clauses 2.1.2.4–5 require disclosure of interest/payment circumstances and fees; clause 2.1.2.9 requires periodic statements including movements, interest, fees and closing balance. Applicability depends on the institution/customer/product. These provisions do not establish this exercise's cutoff, rounding or fee policy. [CBUAE Consumer Protection Standards, pages 9–10](#).

Control before live use: require independently approved backdated adjustments with an immutable cause/reference, both dates and a preview of affected fees/interest; reconcile resulting postings and statements before release. Closed-period changes follow the bank's approved process. This is a recommended control, not a claim that the cited regulation prescribes this exact workflow. Durable recovery and a contractual calendar remain separate prerequisites.